

Python Code

```

1  #
   =====
2  # INGENIEURINFORMATIK 3 - RBTREE REPOSITORY (32 FUNCTIONS)
3  #
   =====
4  # SCHNELLSUCH-INDEX (Nutze Strg+F für sekundenschnellen Zugriff in der
   Klausur):
5  #
6  # --- KAPITEL 1: DIE TRAVERSIERUNGS-MATRIX (ALLE REIHENFOLGEN x BEIDE
   VARIANTEN) ---
7  # [A1]  -> Inorder: Liste von Keys [inorder_keys]
8  # [A2]  -> Inorder: Callback mit Keys [visit_inorder_keys]
9  # [A3]  -> Inorder: Callback mit Values [visit_inorder_values]
10 # [A4]  -> Preorder: Liste von Keys [preorder_keys]
11 # [A5]  -> Preorder: Liste von Values [get_values_preorder] (Aus dem Ur-PDF)
12 # [A6]  -> Preorder: Callback mit Keys [visit_preorder_keys]
13 # [A7]  -> Postorder: Liste von Keys [get_keys_postorder] (Aus dem Ur-PDF)
14 # [A8]  -> Postorder: Callback mit Keys [visit_post_order] (Aus der
   Altklausur)
15 # [A9]  -> Level-Order: Liste von Keys [levelorder_keys]
16 # [A10] -> Level-Order: Callback mit Keys [visit_level_order] (Aus der
   Altklausur)
17 #
18 # --- KAPITEL 2: METRIKEN, STATISTIKEN & ZÄHLOPERATIONEN ---
19 # [A11] -> Anzahl aller echten Knoten [count_nodes]
20 # [A12] -> Anzahl aller echten Blätter [count_leaves]
21 # [A13] -> Anzahl aller roten Knoten [count_red_nodes]
22 # [A14] -> Anzahl aller schwarzen Knoten (exkl. NIL) [count_black_nodes]
23 # [A15] -> Arithmetischer Durchschnitt aller Schlüssel [average_key]
24 # [A16] -> Summe aller Schlüsselwerte [sum_keys]
25 #
26 # --- KAPITEL 3: BAUMTIEFEN & ROT-SCHWARZ-HOCHSPEZIFISCHE METRIKEN ---
27 # [A17] -> Maximale Baumtiefe / Gesamthöhe [get_max_depth]
28 # [A18] -> Minimale Baumtiefe [get_min_depth]
29 # [A19] -> Schwarzhöhe über den LINKEN Pfad [get_black_height] /
   [get_black_height_left]
30 # [A20] -> Schwarzhöhe über den RECHTEN Pfad [get_black_height_right]
31 # [A21] -> Validierung: Haben ALLE Pfade dieselbe Schwarzhöhe?
   [validate_black_height]
32 # [A22] -> Rothöhe: Maximale rote Knotenfolge auf einem Pfad
   [get_max_consecutive_red]
33 #
34 # --- KAPITEL 4: SUCH- & INTERVALLOPREATIONEN (MIT FEHLERBEHANDLUNGEN) ---
35 # [A23] -> Globales Minimum (ValueError bei Leere) [min]
36 # [A24] -> Globales Maximum (ValueError bei Leere) [max]
37 # [A25] -> Iterative Elementsuche (KeyError bei Fehlen) [find]
38 # [A26] -> Intervall-Suche / Werte dazwischen [between]
39 # [A27] -> Inorder-Nachfolger / Nächstgrößeren Key finden [get_successor]
40 # [A28] -> Inorder-Vorgänger / Nächstkleinere Key finden [get_predecessor]
41 # [A29] -> Keys auf einer exakten Ebene sammeln [get_keys_at_level]

```

```

42 #
43 # --- KAPITEL 5: STRUKTUR-VALIDIERUNG & VERWANDTSCHAFTEN ---
44 # [A30] -> Rot-Eigenschaft prüfen: Keine zwei roten Knoten in Folge
    [check_red_property]
45 # [A31] -> Verwandtschaft: Onkel-Schlüssel eines Keys ermitteln
    [get_uncle_key]
46 # [A32] -> Den gesamten Baum leeren / zurücksetzen [clear]
47 #
    =====
48
49 """
50 KLAUSUR-HINWEIS: Alle Methoden nutzen 'self._root' und berücksichtigen
    'node.is_nil'.
51 Sie sind so eingerückt, dass sie direkt per Copy-Paste in die Klasse passen.
52
53 Für Export: Öffnen der Datei über "Open Folder". In Konsole ./zipUpload.sh
    eingeben.
54 """
55
56 #
    =====
57 # --- KAPITEL 1: DIE TRAVERSIERUNGS-MATRIX ---
58 #
    =====
59
60 # [A1] Inorder: Liste von Keys
61 def inorder_keys(self) -> list:
62     def _rec(node, res):
63         if node and not node.is_nil:
64             _rec(node.left, res)
65             res.append(node.key)
66             _rec(node.right, res)
67     result = []
68     _rec(self._root, result)
69     return result
70
71 # [A2] Inorder: Callback mit Keys
72 def visit_inorder_keys(self, visit) -> None:
73     def _rec(node):
74         if node and not node.is_nil:
75             _rec(node.left)
76             visit(node.key)
77             _rec(node.right)
78     _rec(self._root)
79
80 # [A3] Inorder: Callback mit Values
81 def visit_inorder_values(self, visit) -> None:
82     def _rec(node):
83         if node and not node.is_nil:
84             _rec(node.left)
85             visit(node.value)
86             _rec(node.right)
87     _rec(self._root)
88

```

```

89 # [A4] Preorder: Liste von Keys
90 def preorder_keys(self) -> list:
91     def _rec(node, res):
92         if node and not node.is_nil:
93             res.append(node.key)
94             _rec(node.left, res)
95             _rec(node.right, res)
96     result = []
97     _rec(self._root, result)
98     return result
99
100 # [A5] Preorder: Liste von Values (Aus Original-PDF)
101 def get_values_preorder(self) -> list:
102     def _preorder_rec(node, result: list):
103         if node and not node.is_nil:
104             result.append(node.value)
105             _preorder_rec(node.left, result)
106             _preorder_rec(node.right, result)
107     res = []
108     _preorder_rec(self._root, res)
109     return res
110
111 # [A6] Preorder: Callback mit Keys
112 def visit_preorder_keys(self, visit) -> None:
113     def _rec(node):
114         if node and not node.is_nil:
115             visit(node.key)
116             _rec(node.left)
117             _rec(node.right)
118     _rec(self._root)
119
120 # [A7] Postorder: Liste von Keys (Aus Original-PDF)
121 def get_keys_postorder(self) -> list:
122     def _postorder_rec(node, result: list):
123         if node and not node.is_nil:
124             _postorder_rec(node.left, result)
125             _postorder_rec(node.right, result)
126             result.append(node.key)
127     res = []
128     _postorder_rec(self._root, res)
129     return res
130
131 # [A8] Postorder: Callback mit Keys (Aus Altklausur)
132 def visit_post_order(self, visit) -> None:
133     def _post_order_rec(node):
134         if node and not node.is_nil:
135             _post_order_rec(node.left)
136             _post_order_rec(node.right)
137             visit(node.key)
138     _post_order_rec(self._root)
139
140 # [A9] Level-Order: Liste von Keys
141 def levelorder_keys(self) -> list:
142     if self._root.is_nil:

```

```

143         return []
144     result, queue = [], [self._root]
145     while queue:
146         curr = queue.pop(0)
147         result.append(curr.key)
148         if curr.left and not curr.left.is_nil:
149             queue.append(curr.left)
150         if curr.right and not curr.right.is_nil:
151             queue.append(curr.right)
152     return result
153
154     # [A10] Level-Order: Callback mit Keys (Aus Altklausur)
155     def visit_level_order(self, visit) -> None:
156         if self._root.is_nil:
157             return
158         queue = [self._root]
159         while len(queue) > 0:
160             current = queue.pop(0)
161             visit(current.key)
162             if current.left and not current.left.is_nil:
163                 queue.append(current.left)
164             if current.right and not current.right.is_nil:
165                 queue.append(current.right)
166
167     #
168     =====
169     # --- KAPITEL 2: METRIKEN, STATISTIKEN & ZÄHLOPERATIONEN ---
170     #
171     =====
172
173     # [A11] Anzahl aller echten Knoten
174     def count_nodes(self) -> int:
175         def _count_rec(node):
176             if not node or node.is_nil:
177                 return 0
178             return 1 + _count_rec(node.left) + _count_rec(node.right)
179         return _count_rec(self._root)
180
181     # [A12] Anzahl aller echten Blätter
182     def count_leaves(self) -> int:
183         def _count_rec(node) -> int:
184             if not node or node.is_nil:
185                 return 0
186             if (not node.left or node.left.is_nil) and (not node.right or
node.right.is_nil):
187                 return 1
188             return _count_rec(node.left) + _count_rec(node.right)
189         return _count_rec(self._root)
190
191     # [A13] Anzahl aller roten Knoten
192     def count_red_nodes(self) -> int:
193         def _rec(node) -> int:
194             if not node or node.is_nil:

```

```

194         return 0
195         curr = 1 if not node.is_black else 0
196         return curr + _rec(node.left) + _rec(node.right)
197     return _rec(self._root)
198
199 # [A14] Anzahl aller schwarzen Knoten (exklusive NIL)
200 def count_black_nodes(self) -> int:
201     def _rec(node) -> int:
202         if not node or node.is_nil:
203             return 0
204             curr = 1 if node.is_black else 0
205             return curr + _rec(node.left) + _rec(node.right)
206     return _rec(self._root)
207
208 # [A15] Arithmetischer Durchschnitt aller Schlüssel
209 def average_key(self) -> float:
210     keys = self.inorder_keys()
211     if not keys:
212         return 0.0
213     return sum(keys) / len(keys)
214
215 # [A16] Summe aller Schlüsselwerte
216 def sum_keys(self) -> int:
217     return sum(self.inorder_keys())
218
219
220 #
=====
221 # --- KAPITEL 3: BAUMTIEFEN & ROT-SCHWARZ-HOCHSPEZIFISCHE METRIKEN ---
222 #
=====
223
224 # [A17] Maximale Baumtiefe / Gesamthöhe
225 def get_max_depth(self) -> int:
226     def _depth_rec(node) -> int:
227         if not node or node.is_nil:
228             return 0
229             return 1 + max(_depth_rec(node.left), _depth_rec(node.right))
230     return _depth_rec(self._root)
231
232 # [A18] Minimale Baumtiefe (Kürzester Pfad zu einem NIL-Blatt)
233 def get_min_depth(self) -> int:
234     def _rec(node) -> int:
235         if not node or node.is_nil:
236             return 0
237             return 1 + min(_rec(node.left), _rec(node.right))
238     return _rec(self._root)
239
240 # [A19] Schwarzhöhe über den LINKEN Pfad
241 def get_black_height(self) -> int:
242     height = 0
243     current = self._root
244     while current is not None:
245         if current.is_black:

```

```

246         height += 1
247     if current.is_nil:
248         break
249     current = current.left
250     return height
251
252 # [A20] Schwarzhöhe über den RECHTEN Pfad
253 def get_black_height_right(self) -> int:
254     height = 0
255     current = self._root
256     while current is not None:
257         if current.is_black:
258             height += 1
259         if current.is_nil:
260             break
261         current = current.right
262     return height
263
264 # [A21] Validierung: Haben ALLE Pfade dieselbe Schwarzhöhe?
265 def validate_black_height(self) -> bool:
266     def _check(node):
267         if not node or node.is_nil:
268             return 1 # Ein NIL-Blatt ist schwarz und zählt als 1
269         left_h = _check(node.left)
270         right_h = _check(node.right)
271         if left_h == -1 or right_h == -1 or left_h != right_h:
272             return -1
273         return left_h + (1 if node.is_black else 0)
274     return _check(self._root) != -1
275
276 # [A22] Rothöhe: Maximale aufeinanderfolgende rote Knoten auf einem Pfad
277 def get_max_consecutive_red(self) -> int:
278     def _rec(node, current_run):
279         if not node or node.is_nil:
280             return current_run
281         if not node.is_black:
282             current_run += 1
283         else:
284             current_run = 0
285         return max(current_run, _rec(node.left, current_run),
286 _rec(node.right, current_run))
287     return _rec(self._root, 0)
288
289 #
290 # --- KAPITEL 4: SUCH- & INTERVALLOPREATIONEN ---
291 #
292
293 # [A23] Globales Minimum
294 def min(self) -> int:
295     if self._root.is_nil:
296         raise ValueError("Der Baum ist leer.")

```

```

297         current = self._root
298         while current.left and not current.left.is_nil:
299             current = current.left
300         return current.key
301
302     # [A24] Globales Maximum
303     def max(self) -> int:
304         if self._root.is_nil:
305             raise ValueError("Der Baum ist leer.")
306         current = self._root
307         while current.right and not current.right.is_nil:
308             current = current.right
309         return current.key
310
311     # [A25] Iterative Elementsuche mit exakter Altklausur-Fehlerbehandlung
312     def find(self, key: int) -> any:
313         current = self._root
314         while current and not current.is_nil:
315             if key == current.key:
316                 return current.value
317             elif key < current.key:
318                 current = current.left
319             else:
320                 current = current.right
321         raise KeyError(f"Key {key} wurde nicht im Baum gefunden.")
322
323     # [A26] Intervall-Suche / Werte dazwischen [low, high]
324     def between(self, low: int, high: int) -> list:
325         result = []
326         def _between_rec(node):
327             if not node or node.is_nil:
328                 return
329             if node.key > low:
330                 _between_rec(node.left)
331             if low <= node.key <= high:
332                 result.append(node.key)
333             if node.key < high:
334                 _between_rec(node.right)
335         _between_rec(self._root)
336         return result
337
338     # [A27] Symmetrischen Inorder-Nachfolger (Successor) finden
339     def get_successor(self, key: int) -> any:
340         current = self._root
341         successor = None
342         while current and not current.is_nil:
343             if key < current.key:
344                 successor = current
345                 current = current.left
346             elif key > current.key:
347                 current = current.right
348             else:
349                 if current.right and not current.right.is_nil:
350                     curr_right = current.right

```

```

351         while curr_right.left and not curr_right.left.is_nil:
352             curr_right = curr_right.left
353         return curr_right.key
354     break
355     return successor.key if successor else None
356
357 # [A28] Symmetrischen Inorder-Vorgänger (Predecessor) finden
358 def get_predecessor(self, key: int) -> any:
359     current = self._root
360     predecessor = None
361     while current and not current.is_nil:
362         if key < current.key:
363             current = current.left
364         elif key > current.key:
365             predecessor = current
366             current = current.right
367         else:
368             if current.left and not current.left.is_nil:
369                 curr_left = current.left
370                 while curr_left.right and not curr_left.right.is_nil:
371                     curr_left = curr_left.right
372                 return curr_left.key
373             break
374     return predecessor.key if predecessor else None
375
376 # [A29] Schlüssel auf einer exakten Ebene sammeln
377 def get_keys_at_level(self, target_level: int) -> list:
378     result = []
379     def _collect_level_rec(node, current_level):
380         if not node or node.is_nil:
381             return
382         if current_level == target_level:
383             result.append(node.key)
384             return
385         if current_level < target_level:
386             _collect_level_rec(node.left, current_level + 1)
387             _collect_level_rec(node.right, current_level + 1)
388     _collect_level_rec(self._root, 0)
389     return result
390
391
392 #
=====
393 # --- KAPITEL 5: STRUKTUR-VALIDIERUNG & VERWANDTSCHAFTEN ---
394 #
=====
395
396 # [A30] Rot-Schwarz-Eigenschaft prüfen (Red-Property)
397 def check_red_property(self) -> bool:
398     def _check_rec(node) -> bool:
399         if not node or node.is_nil:
400             return True
401         if not node.is_black:

```



```
402         if node.left and not node.left.is_nil and not
node.left.is_black:
403             return False
404         if node.right and not node.right.is_nil and not
node.right.is_black:
405             return False
406         return _check_rec(node.left) and _check_rec(node.right)
407     return _check_rec(self._root)
408
409 # [A31] Struktur-Hilfe: Onkel-Schlüssel eines Keys ermitteln
410 def get_uncle_key(self, key: int) -> any:
411     current = self._root
412     parent = None
413     grandparent = None
414     while current and not current.is_nil and current.key != key:
415         grandparent = parent
416         parent = current
417         if key < current.key:
418             current = current.left
419         else:
420             current = current.right
421     if not current or current.is_nil or not parent or not grandparent:
422         return None
423     if grandparent.left == parent:
424         uncle = grandparent.right
425     else:
426         uncle = grandparent.left
427     return uncle.key if uncle and not uncle.is_nil else None
428
429 # [A32] Den gesamten Baum leeren / zurücksetzen
430 def clear(self) -> None:
431     self._root = RBTreeNode.nil()
```